Topic 29 Juggling the Real World

> *Things don't just happen; they are made to happen.*
>
> *John F. Kennedy*

In the old days, when your authors still had their boyish good looks, computers were not particularly flexible. We'd typically organize the way we interacted with them based on their limitations.

Today, we expect more: computers have to integrate into *our* world, not the other way around. And our world is messy: things are constantly happening, stuff gets moved around, we change our minds, ... And the applications we write somehow have to work out what to do.

This section is all about writing these responsive applications.

We'll start off with the concept of an *event*.

Events

An *event* represents the availability of information. It might come from the outside world: a user clicking a button, or a stock quote update. It might be internal: the result of a calculation is ready, a search finishes. It can even be something as trivial as fetching the next element in a list.

Whatever the source, if we write applications that respond to events, and adjust what they do based on those events, those applications will work better in the real world. Their users will find

them to be more interactive, and the applications themselves will make better use of resources.

But how can we write these kinds of applications? Without some kind of strategy, we'll quickly find ourselves confused, and our applications will be a mess of tightly coupled code.

Let's look at four strategies that help.

1. Finite State Machines
2. The Observer Pattern
3. Publish/Subscribe
4. Reactive Programming and Streams

Finite State Machines

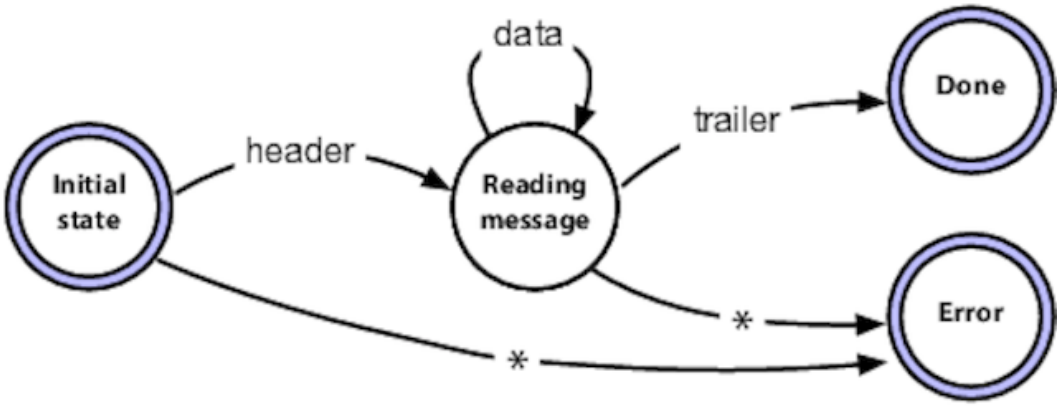
Dave finds that he writes code using a Finite State Machine (FSM) just about every week. Quite often, the FSM implementation will be just a couple of lines of code, but those few lines help untangle a whole lot of potential mess.

Using an FSM is trivially easy, and yet many developers shy away from them. There seems to be a belief that they are difficult, or that they only apply if you're working with hardware, or that you need to use some hard-to-understand library. None of these are true.

The Anatomy of a Pragmatic FSM

A state machine is basically just a specification of how to handle events. It consists of a set of states, one of which is the *current state*. For each state, we list the events that are significant to that state. For each of those events, we define the new current state of the system.

For example, we may be receiving multipart messages from a websocket. The first message is a header. This is followed by any number of data messages, followed by a trailing message. This could be represented as an FSM like this:



We start in the “Initial state.” If we receive a header message, we *transition* to the “Reading message” state. If we receive anything else while we’re in the initial state (the line labeled with an asterisk) we transition to the “Error” state and we’re done.

While we’re in the “Reading message” state, we can accept either data messages, in which case we continue reading in the same state, or we can accept a trailer message, which transitions us to the “Done” state. Anything else causes a transition to the error state.

The neat thing about FSMs is that we can express them purely as data. Here’s a table representing our message parser:

State	Events			
	Header	Data	Trailer	Other
Initial	Reading	Error	Error	Error
Reading	Error	Reading	Done	Error

The rows in the table represent the states. To find out what to do when an event occurs, look up the row for the current state, scan along for the column representing the event, the contents of that cell are the new state.

The code that handles it is equally simple:

event/simple_fsm.rb

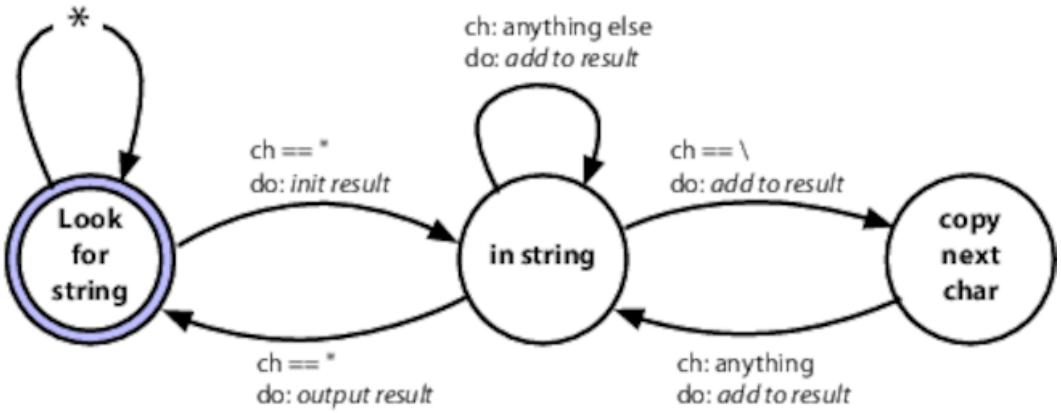
```
1:  TRANSITIONS = {
 -    initial: {header: :reading},
 -    reading: {data: :reading, trailer: :done},
 -  }
5:
 -  state = :initial
 -
 -  while state != :done && state != :error
 -    msg = get_next_message()
      state = TRANSITIONS[state][msg.msg_type] || :error
10:

 -  end
```

The code that implements the transitions between states is on line 10. It indexes the transition table using the current state, and then indexes the transitions for that state using the message type. If there is no matching new state, it sets the state to `:error`.

Adding Actions

A pure FSM, such as the one we were just looking at, is an event stream parser. Its only output is the final state. We can beef it up by adding actions that are triggered on certain transitions.

For example, we might need to extract all of the strings in a source file. A string is text between quotes, but a backslash in a string escapes the next character, so `"Ignore \"quotes\""` is a single string. Here's an FSM that does this:



This time, each transition has two labels. The top one is the event that triggers it, and the bottom one is the action to take as we move between states.

We'll express this in a table, as we did last time. However, in this case each entry in the table is a two-element list containing the next state and the name of an action:

event/strings_fsm.rb

```
TRANSITIONS = {

  # current        new state        action to take
  #-------------------------------------------------------

  look_for_string: {
    '"'      => [ :in_string,       :start_new_string ],
    :default => [ :look_for_string, :ignore ],
  },

  in_string: {
    '"'      => [ :look_for_string, :finish_current_string ],
    '\\'     => [ :copy_next_char,  :add_current_to_string ],
    :default => [ :in_string,       :add_current_to_string ],
  },
```

```
    copy_next_char: {
      :default => [ :in_string,       :add_current_to_string ],
    },
  }
```

We've also added the ability to specify a default transition, taken if the event doesn't match any of the other transitions for this state.

Now let's look at the code:

event/strings_fsm.rb

```
state = :look_for_string
result = []

while ch = STDIN.getc
  state, action = TRANSITIONS[state][ch] || TRANSITIONS[state][
:default]
  case action
  when :ignore
  when :start_new_string
    result = []
  when :add_current_to_string
    result << ch
  when :finish_current_string
    puts result.join
  end
end
```

This is similar to the previous example, in that we loop through the events (the characters in the input), triggering transitions. But it does more than the previous code. The result of each transition is both a new state and the name of an action. We use the action name to select the code to run before we go back around the loop.

This code is very basic, but it gets the job done. There are many other variants: the transition table could use anonymous functions or function pointers for the actions, you could wrap the code that implements the state machine in a separate class, with its own state, and so on.

There's nothing to say that you have to process all the state transitions at the same time. If you're going through the steps to sign up a user on your app, there's likely to be a number of transitions as they enter their details, validate their email, agree to the 107 different legislated warnings that online apps must now give, and so on. Keeping the state in external storage, and using it to drive a state machine, is a great way to handle these kind of workflow requirements.

State Machines Are a Start

State machines are underused by developers, and we'd like to encourage you to look for opportunities to apply them. But they don't solve all the problems associated with events. So let's move on to some other ways of looking at the problems of juggling events.

The Observer Pattern

In the *observer pattern* we have a source of events, called the *observable* and a list of clients, the *observers*, who are interested in those events.

An observer registers its interest with the observable, typically by passing a reference to a function to be called. Subsequently, when the event occurs, the observable iterates down its list of observers and calls the function that each passed it. The event is given as a parameter to that call.

Here's a simple example in Ruby. The `Terminator` module is used to terminate the application. Before it does so, however, it notifies all its observers that the application is going to exit.[39] They might use this notification to tidy up temporary resources, commit data, and so on:

event/observer.rb

```
module Terminator
  CALLBACKS = []

  def self.register(callback)
    CALLBACKS << callback
  end

  def self.exit(exit_status)
    CALLBACKS.each { |callback| callback.(exit_status) }
    exit!(exit_status)
  end
end

Terminator.register(-> (status) { puts "callback 1 sees #{status}"
})
Terminator.register(-> (status) { puts "callback 2 sees #{status}"
})

Terminator.exit(99)
```

```
$ ruby event/observer.rb
callback 1 sees 99
callback 2 sees 99
```

There's not much code involved in creating an observable: you push a function reference onto a list, and then call those functions when the event occurs. This is a good example of when *not* to use a library.

The observer/observable pattern has been used for decades, and it has served us well. It is particularly prevalent in user interface systems, where the callbacks are used to inform the application that some interaction has occurred.

But the observer pattern has a problem: because each of the observers has to register with the observable, it introduces coupling. In addition, because in the typical implementation the callbacks are handled inline by the observable, synchronously, it can introduce performance bottlenecks.

This is solved by the next strategy, Publish/Subscribe.

Publish/Subscribe

Publish/Subscribe (pubsub) generalizes the observer pattern, at the same time solving the problems of coupling and performance.

In the pubsub model, we have *publishers* and *subscribers*. These are connected via channels. The channels are implemented in a separate body of code: sometimes a library, sometimes a process, and sometimes a distributed infrastructure. All this implementation detail is hidden from your code.

Every channel has a name. Subscribers register interest in one or more of these named channels, and publishers write events to them. Unlike the observer pattern, the communication between the publisher and subscriber is handled outside your code, and is potentially asynchronous.

Although you could implement a very basic pubsub system yourself, you probably don’t want to. Most cloud service providers have pubsub offerings, allowing you to connect applications

around the world. Every popular language will have at least one pubsub library.

Pubsub is a good technology for decoupling the handling of asynchronous events. It allows code to be added and replaced, potentially while the application is running, without altering existing code. The downside is that it can be hard to see what is going on in a system that uses pubsub heavily: you can't look at a publisher and immediately see which subscribers are involved with a particular message.

Compared to the observer pattern, pubsub is a great example of reducing coupling by abstracting up through a shared interface (the channel). However, it is still basically just a message passing system. Creating systems that respond to combinations of events will need more than this, so let's look at ways we can add a time dimension to event processing.

Reactive Programming, Streams, and Events

If you've ever used a spreadsheet, then you'll be familiar with *reactive programming*. If a cell contains a formula which refers to a second cell, then updating that second cell causes the first to update as well. The values *react* as the values they use change.

There are many frameworks that can help with this kind of data-level reactivity: in the realm of the browser React and Vue.js are current favorites (but, this being JavaScript, this information will be out-of-date before this book is even printed).

It's clear that events can also be used to trigger reactions in code, but it isn't necessarily easy to plumb them in. That's where *streams* come in.

Streams let us treat events as if they were a collection of data. It's as if we had a list of events, which got longer when new events arrive. The beauty of that is that we can treat streams just like any other collection: we can manipulate, combine, filter, and do all the other data-ish things we know so well. We can even combine event streams and regular collections. And streams can be asynchronous, which means your code gets the opportunity to respond to events as they arrive.

The current *de facto* baseline for reactive event handling is defined on the site http://reactivex.io, which defines a language-agnostic set of principles and documents some common implementations. Here we'll use the RxJs library for JavaScript.

Our first example takes two streams and zips them together: the result is a new stream where each element contains one item from the first input stream and one item from the other. In this case, the first stream is simply a list of five animal names. The second stream is more interesting: it's an interval timer which generates an event every 500ms. Because the streams are zipped together, a result is only generated when data is available on both, and so our result stream only emits a value every half second:

event/rx0/index.js

```
import * as Observable from 'rxjs'
import { logValues }   from "../rxcommon/logger.js"

let animals  = Observable.of("ant", "bee", "cat", "dog", "elk")
let ticker   = Observable.interval(500)

let combined = Observable.zip(animals, ticker)

combined.subscribe(next => logValues(JSON.stringify(next)))
```

This code uses a simple logging function[40] which adds items to a list in the browser window. Each item is timestamped with the time in milliseconds since the program started to run. Here's what it shows for our code:

```
502 ms
  ["ant",0]

1002 ms
  ["bee",1]

1502 ms
  ["cat",2]

2002 ms
  ["dog",3]

2502 ms
  ["elk",4]
```

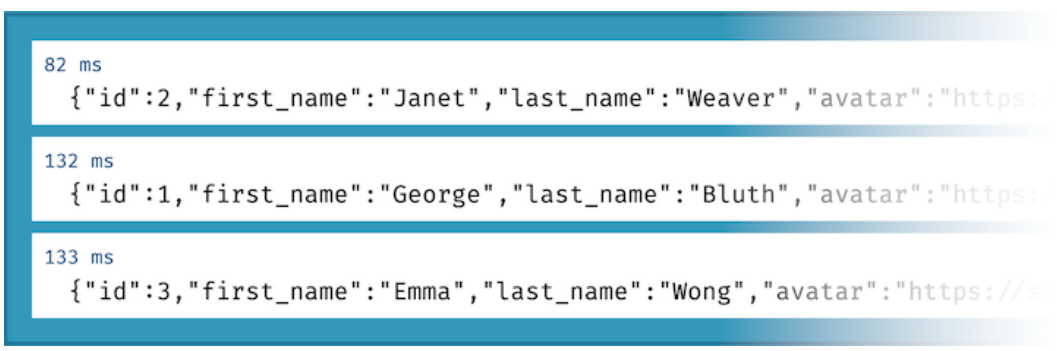
Notice the timestamps: we're getting one event from the stream every 500ms. Each event contains a serial number (created by the `interval` observable) and the name of the next animal from the list. Watching it live in a browser, the log lines appear at every half second.

Event streams are normally populated as events occur, which implies that the observables that populate them can run in parallel. Here's an example that fetches information about users from a remote site. For this we'll use https://reqres.in, a public site that provides an open REST interface. As part of its API, we can fetch data on a particular (fake) user by performing a GET request to `users/«id»`. Our code fetches the users with the IDs 3, 2, and 1:

event/rx1/index.js

```
import * as Observable from 'rxjs'
import { mergeMap }    from 'rxjs/operators'
import { ajax }        from 'rxjs/ajax'
import { logValues }   from "../rxcommon/logger.js"

let users = Observable.of(3, 2, 1)

let result = users.pipe(
  mergeMap((user) => ajax.getJSON(`https://reqres.in/api/users/
${user}`))
)

result.subscribe(
  resp => logValues(JSON.stringify(resp.data)),
  err  => console.error(JSON.stringify(err))
)
```

The internal details of the code are not too important. What's exciting is the result, shown in the following screenshot:



Look at the timestamps: the three requests, or three separate streams, were processed in parallel, The first to come back, for id 2, took 82ms, and the next two came back 50 and 51ms later.

Streams of Events Are Asynchronous Collections

In the previous example, our list of user IDs (in the observable **users**) was static. But it doesn't have to be. Perhaps we want to

collect this information when people log in to our site. All we have to do is to generate an observable event containing their user ID when their session is created, and use that observable instead of the static one. We'd then be fetching details about the users as we received these IDs, and presumably storing them somewhere.

This is a very powerful abstraction: we no longer need to think about time as being something we have to manage. Event streams unify synchronous and asynchronous processing behind a common, convenient API.

Events Are Ubiquitous

Events are everywhere. Some are obvious: a button click, a timer expiring. Other are less so: someone logging in, a line in a file matching a pattern. But whatever their source, code that's crafted around events can be more responsive and better decoupled than its more linear counterpart.

Related Sections Include

- Topic 28, *Decoupling*
- Topic 36, *Blackboards*

Exercises

Exercise 19 (possible answer)

In the FSM section we mentioned that you could move the generic state machine implementation into its own class. That class would probably be initialized by passing in a table of transitions and an initial state.

Try implementing the string extractor that way.

Exercise 20 (possible answer)

Which of these technologies (perhaps in combination) would be a good fit for the following situations:

- If you receive three *network interface down* events within five minutes, notify the operations staff.
- If it is after sunset, and there is motion detected at the bottom of the stairs followed by motion detected at the top of the stairs, turn on the upstairs lights.
- You want to notify various reporting systems that an order was completed.
- In order to determine whether a customer qualifies for a car loan, the application needs to send requests to three backend services and wait for the responses.